

Intuition

Suppose you start working on a project, it's a long project and you are making changes to the project on a daily basis.

Now, one day while making changes to a function, you realised that the previous version of this function made more sense and that you have want to go back to that version.

You are now lost because you don't remember what that function exactly was.

→ For this exact reason, there is a need to version our projects and code.

What is Git

Git is a version control system that lets you keep track of all the modifications you make to your code.

This means if a new feature is causing some errors in your code, you can always roll back to a previous stable version.

But Git isn't any VCS, it's a **distributed VCS** which means that every collaborator of the project will have a history of the changes made on their local machine.

So, people can work on different features of the project without having to communicate with each other or the server hosting the project.

It is written in C.

What is GitHub?

GitHub is a widely used platform for version control that uses Git at its core.

It lets you host the remote version of your project from where all the collaborators can have access to it.

Not just your own team members but any member of GitHub can contribute to your code (if it is a public project and you accept the changes proposed.)

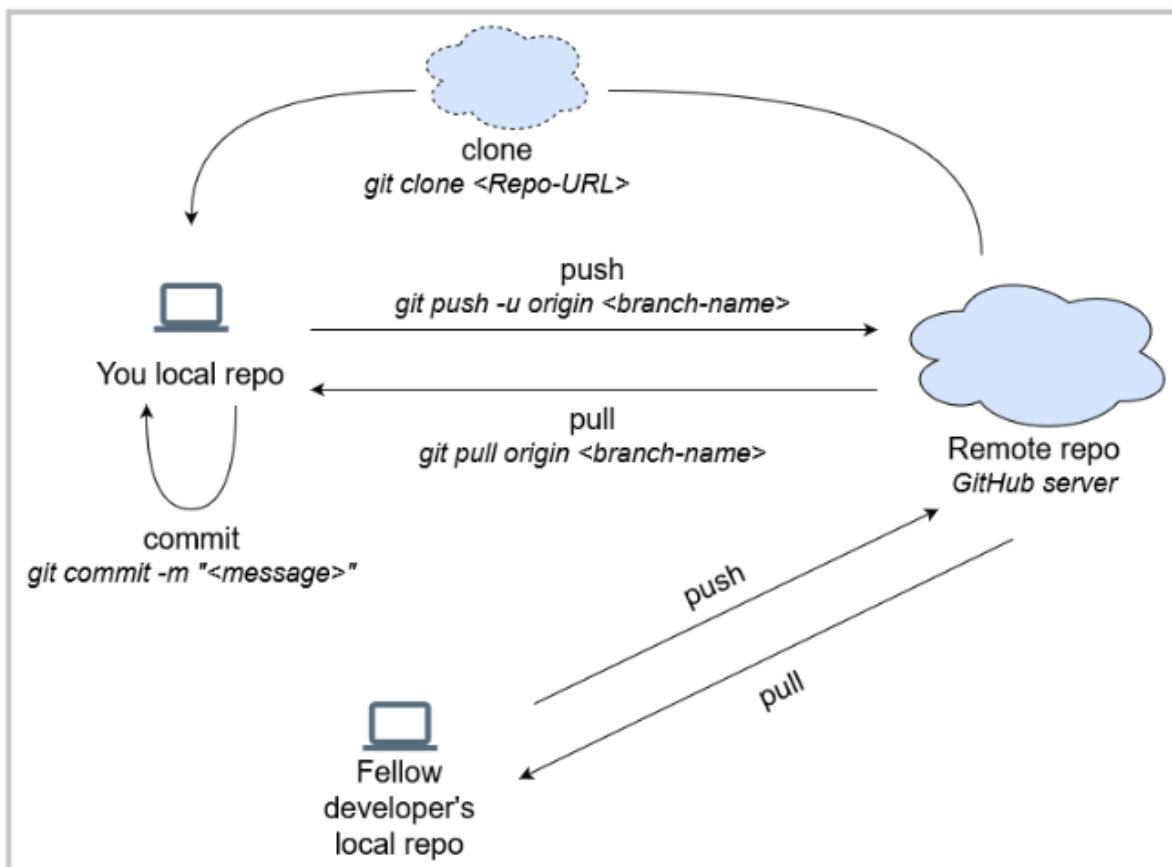
Social Platform for coders: It also serves as a platform where people can find plethora of open-source projects with their codes. You can collaborate on amazing projects and have discussions

Git Terminologies: What is a repository?

A repository is a folder that contains all the project files and the history of the revisions made to each file.

There are two repositories of your project that you will work with throughout the lifetime of your project:

1. **Remote repository:** contains your project that can be accessed from anywhere and by anyone. It lives on one of the hosting platforms like GitHub, Bitbucket or Gitlabs.
2. **Local repository:** A copy of the remote repository that resides on your local machine. All the changes you make will be saved in your local repository.



Actual Project Setup

1. Create a repository using GitHub Desktop / Can also show that they can initialise a git repository using CLI.
2. Start a folder
3. Add a README.md, .ipynb and .py, and data folder in the repo.
4. Initialize a git repository.
5. Adding to staging area.
6. Commit - what does it mean to commit code?
7. What does it mean to push the code.
8. What does it mean to pull the code from remote?

Branching (A, B and conflict)

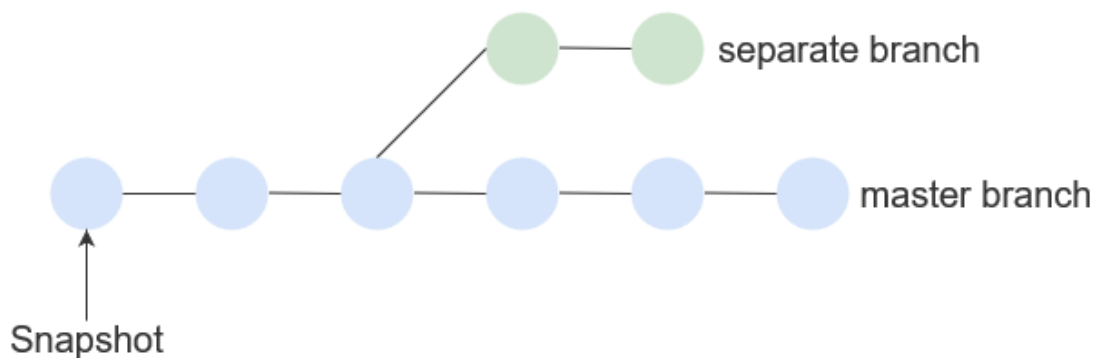
If you want to work on a new feature or analysis or fix a bug in the main project, you don't want to mess up the original working code, so what would you do in that case?

Can you always copy the entire code and make changes there and come back to the main project with the changes?

There are still chances of messing up while copying and pasting.

→ **To help us with this, Git has a fundamental feature of branching.**

Branches are like reference to a commit.



- I can draw out a new branch from the main branch and make the changes there.

- Once I'm sure everything is finished and working successfully, I can raise a PR to merge it back to the main branch.
- And lastly, I delete the branch.

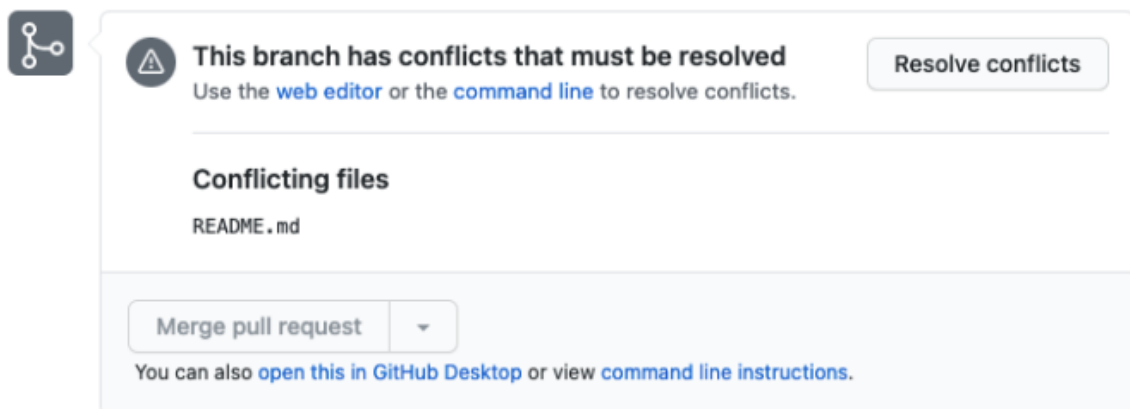
Collaboration

While collaborating with other team members, there can be many conflicts and we may need to resolve conflicts.

Let's say there are two branches (**a** and **b**) that were created from the **main** branch.

Here's what we're going to try and simulate:

1. Developer A and B fork the **main** branch to make some changes
2. Developer A makes a change and submits a PR to the **main** branch.
3. Developer B makes a change to the same line as Developer A and submits a PR to **main**.
4. Merge the first PR to the main branch.
5. We have a merge conflict for the 2nd PR now since both developers altered the same line.
6. We'll have to choose which version of the code to keep and resolve the conflict.



Project Cloning

Project Contribution - Fork, branch out and submit a PR

<https://docs.github.com/en/get-started/quickstart/contributing-to-projects> Best practices

There so many different works to work with git and sometimes it can become quickly unruly when fellow developers follow different practices. Here are a few, widely accepted, best practices when it comes to working with commits and branches.

Commits

- Commit often such that each commit has a clear associated change which you can approve / rollback.
- Try and [squash](#) commits if you have multiple before pushing to the remote host.
- Avoid monolithic commits (even if you regularly stash and rebase) because it can cause many components to break and creates a code review nightmare.
- Attach meaningful messages to commits so developers know exactly what the PR entails.
- Use tags to represent meaningful and stable releases of your application.

Tags

```
git tag -a v0.1 -m "initial release"
```

- Don't delete commit history (reset), instead use [revert](#) to rollback and provide reasoning.